Practical File

of

Object Oriented Language

Course Code: CSEG1044

School of Computer Science

Submitted By

Student Name: Suryansh Mishra

SAP ID: 590012069

Course: B.Tech CSE Core

Semester: 4th

Batch: 4

Academic Year: 2025-26

Submitted To

Mr. Piyush Bagla

/*5.1: Employee Performance Matrix Analyzer

Problem Statement

A company stores employee performance ratings in a **2D array**, where each row represents an employee and each column represents performance scores in different quarters.

Write a Java program to:

1. Store performance scores of **N employees across M quarters**.
2. Find:
    - Employee with **highest average performance**
    - Quarter with **maximum overall performance**
    - Employees whose performance is **consistently improving**
3. Display sorted employee list based on overall performance.

Hint:

Employee Name	Quarter 1	Quarter 2	Quarter 3	Quarter 4
A	70	80	55	60
B	50	60	70	80
C	70	60	90	30

Input Example

Enter number of employees: 3

Enter number of quarters: 4

Expected Output

Top Performer: Employee 2

Best Performing Quarter: Quarter 2

Improving Employees: Employee 2

Sorted Performance Ranking:

Employee 3

Employee 1

Employee 2

*/

Code :-

```
import java.util.*;
public class Experiment41 {
    static Scanner sc;
    static int n, m;
    static String[] names;
    static int[][] scores;
    static int[] sum;
    static double[] avg;
    static int topIdx;
    static int bestQuarter;
    static List<Integer> improving;
    static Integer[] idx;
    public static void main(String[] args) {
        sc = new Scanner(System.in);
        input();
        average();
        output();
        sc.close();
    }
    static void input() {
        System.out.print("Enter number of employees: ");
        n = sc.nextInt();
        System.out.print("Enter number of quarters: ");
        m = sc.nextInt();
        sc.nextLine();
        names = new String[n];
        for (int i = 0; i < n; i++) {
            System.out.print("Enter name of employee " + (i + 1) + " (or press
Enter to use 'Employee " + (i + 1) + "'): ");
            String line = sc.nextLine().trim();
            names[i] = line.isEmpty() ? ("Employee " + (i + 1)) : line;
        }
        scores = new int[n][m];
        for (int i = 0; i < n; i++) {
            System.out.println("Enter " + m + " scores for " + names[i] + "
(separated by spaces):");
```

```
            for (int j = 0; j < m; j++) {
                scores[i][j] = sc.nextInt();
            }
            sc.nextLine();
        }
    }
    static void average() {
        sum = new int[n];
        for (int i = 0; i < n; i++) {
            int s = 0;
            for (int j = 0; j < m; j++) s += scores[i][j];
            sum[i] = s;
        }
        avg = new double[n];
        for (int i = 0; i < n; i++) avg[i] = (double) sum[i] / m;
        topIdx = 0;
        for (int i = 1; i < n; i++) if (avg[i] > avg[topIdx]) topIdx = i;
        int[] quarterSum = new int[m];
        bestQuarter = 0;
        for (int j = 0; j < m; j++) {
            int s = 0;
            for (int i = 0; i < n; i++) s += scores[i][j];
            quarterSum[j] = s;
        }
        for (int j = 1; j < m; j++) if (quarterSum[j] > quarterSum[bestQuarter])
bestQuarter = j;
        improving = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            boolean inc = true;
            for (int j = 0; j < m - 1; j++) {
                if (!(scores[i][j] < scores[i][j + 1])) {
                    inc = false;
                    break;
                }
            }
            if (inc) improving.add(i);
        }
        idx = new Integer[n];
        for (int i = 0; i < n; i++) idx[i] = i;
        Arrays.sort(idx, (a, b) -> Integer.compare(sum[b], sum[a]));
    }
    static void output() {
        System.out.println();
        printInputTable();
        System.out.println("Top Performer: " + names[topIdx]);
```

```java
        System.out.println("Best Performing Quarter: Quarter " + (bestQuarter +
1));
        System.out.print("Improving Employees: ");
        if (improving.isEmpty()) {
            System.out.println("None");
        } else {
            for (int k = 0; k < improving.size(); k++) {
                if (k > 0) System.out.print(", ");
                System.out.print(names[improving.get(k)]);
            }
            System.out.println();
        }
        System.out.println();
        System.out.println("Sorted Performance Ranking:");
        for (int i = 0; i < n; i++) {
            System.out.println(names[idx[i]]);
        }
    }

    static void printInputTable() {
        System.out.println();
        System.out.println("Input Table:");
        System.out.printf("%-16s", "Employee Name");
        for (int j = 0; j < m; j++) {
            System.out.printf("Q%-8d", j + 1);
        }
        System.out.println();
        for (int i = 0; i < n; i++) {
            System.out.printf("%-16s", names[i]);
            for (int j = 0; j < m; j++) {
                System.out.printf("%-9d", scores[i][j]);
            }
            System.out.println();
        }
    }
}
```

Output :-

```
PS C:\Users\surya\OneDrive\Desktop\University\SEM4\OOPS\experiment4> cd "c:\Users\surya\OneDrive\Desktop\University\SEM4\OOPS\experiment4\" ; if ($
?) { javac Experiment41.java } ; if ($?) { java Experiment41 }
Enter number of employees: 3
Enter number of quarters: 4
Enter name of employee 1 (or press Enter to use 'Employee 1'): A
Enter name of employee 2 (or press Enter to use 'Employee 2'): B
Enter name of employee 3 (or press Enter to use 'Employee 3'): C
Enter 4 scores for A (separated by spaces):
70 80 55 60
Enter 4 scores for B (separated by spaces):
50 60 70 80
Enter 4 scores for C (separated by spaces):
70 60 90 30


Input Table:
Employee Name   Q1       Q2       Q3       Q4
A               70       80       55       60
B               50       60       70       80
C               70       60       90       30
Top Performer: A
Best Performing Quarter: Quarter 3
Improving Employees: B

Sorted Performance Ranking:
A
B
C
```

/*6.2. A company wants to analyze customer feedback text automatically.

Write a Java program to:

1. Accept a paragraph from user.
2. Perform the following operations using **Java String methods**:
    - Count total words and sentences
    - Identify most frequently used word
    - Replace all negative words from a given list with "***"
    - Check if feedback contains promotional keywords
    - Display reversed sentence order

Constraints

- Must use Java built-in String functions
- Case insensitive processing required

Input Example

Enter feedback:

The product quality is bad but delivery was fast.

Expected Output

Total Words: 8

Most Frequent Word: product

Filtered Feedback: The product quality is *** but delivery was fast.

Contains Promotion Keyword: No

Reversed Sentence: fast was delivery but *** is quality product The

*/

<u>Code :-</u>

```
import java.util.*;
import java.util.regex.*;
public class Experiment42 {
    public static void main(String[] args) {
        try (Scanner sc = new Scanner(System.in)) {
            System.out.println("Enter feedback:");
            String feedback = sc.nextLine().trim();
            if (feedback.isEmpty()) {
                System.out.println("No feedback provided.");
                return;
            }
        String[] sentenceParts = feedback.split("[.!?]+\s*");
        int sentenceCount = 0;
        for (String s : sentenceParts) if (!s.trim().isEmpty()) sentenceCount++;
        Pattern wordPattern = Pattern.compile("\\b[\\w']+\\b");
        Matcher wm = wordPattern.matcher(feedback);
        List<String> wordsInOrder = new ArrayList<>();
        while (wm.find()) {
            wordsInOrder.add(wm.group());
        }
        int wordCount = wordsInOrder.size();
        Set<String> stopwords = new HashSet<>(Arrays.asList(
            "the","is","a","an","and","but","or","was","were","in","on","at","by"
,"for","to","of","with","this","that","these","those","it","its","as","are","be",
"have","has","had",
"not","from","they","their","them","he","she","his","her","you","your","yours","w
```

```
e","our","ours","i","me","my","mine","do","does","did","so","if","then","there","
when","where","who","whom"
        ));
        Map<String, Integer> freq = new LinkedHashMap<>();
        for (String w : wordsInOrder) {
            String lw = w.toLowerCase();
            freq.put(lw, freq.getOrDefault(lw, 0) + 1);
        }
        String mostFreq = null;
        int best = 0;
        for (String w : freq.keySet()) {
            if (stopwords.contains(w)) continue;
            int f = freq.get(w);
            if (f > best) { best = f; mostFreq = w; }
        }
        if (mostFreq == null && !wordsInOrder.isEmpty()) {
            mostFreq = wordsInOrder.get(0).toLowerCase();
        }
        String[] negatives =
{"bad","poor","terrible","awful","disappointing","unsatisfactory","horrible",
"worst","negative","dissatisfied","unhappy","subpar","lacking","deficient","flawe
d","inadequate",
"unacceptable","regret","complain","complaint","issue","problem","frustrated","an
gry","upset","annoyed","displeased",
"unfortunate","miserable","depressing","dreadful","abysmal","atrocious","appallin
g","ghastly","heinous","horrendous","nasty","offensive","repulsive","rotten","sic
kening","terrifying","tragic","vile",
"wretched","worthless","unbearable","unpleasant","unspeakable","vicious","wicked"
,"worse",
"worst","abominable","adverse","atrocious","awful","dreadful","egregious","ghastl
y","heinous","horrendous","nasty","offensive","repulsive","rotten","sickening","t
errifying","tragic","vile",
"wretched","worthless","unbearable","unpleasant","unspeakable","vicious","wicked"
,"worse", "worst"};
        String filtered = feedback;
        for (String neg : negatives) {
            Pattern p = Pattern.compile("\\b" + Pattern.quote(neg) + "\\b",
Pattern.CASE_INSENSITIVE);
            Matcher m = p.matcher(filtered);
            filtered = m.replaceAll("***");
        }
        String[] promos = {"sale","discount","offer","buy now","free","promo",
"deal", "limited time", "exclusive", "save", "clearance", "bargain", "coupon",
"special", "flash sale", "bundle", "gift with purchase", "last chance",
"doorbuster", "price drop", "mark down", "clearance", "best price", "lowest
```

```
price", "money back", "guarantee", "risk free", "no obligation", "try it now",
"act now", "don't miss", "while supplies last", "limited stock", "exclusive
offer", "vip", "members only", "early access", "pre-order", "new arrival", "hot
deal"};
        boolean hasPromo = false;
        for (String pk : promos) {
            Pattern p = Pattern.compile("\\b" + Pattern.quote(pk) + "\\b",
Pattern.CASE_INSENSITIVE);
            if (p.matcher(feedback).find()) { hasPromo = true; break; }
        }
        String[] tokens = filtered.split("\\s+");
        List<String> cleanTokens = new ArrayList<>();
        for (String t : tokens) {
            String cleaned =
t.replaceAll("^[^\\p{L}\\p{Nd}]+|[^\\p{L}\\p{Nd}]+$", "");
            if (!cleaned.isEmpty()) cleanTokens.add(cleaned);
        }
        Collections.reverse(cleanTokens);
        String reversed = String.join(" ", cleanTokens);
        System.out.println();
        System.out.println("Total Sentences: " + sentenceCount);
        System.out.println();
        System.out.println("Total Words: " + wordCount);
        System.out.println();
        System.out.println("Most Frequent Word: " + (mostFreq == null ? "" :
mostFreq));
        System.out.println();
        System.out.println("Filtered Feedback: " + filtered);
        System.out.println();
        System.out.println("Contains Promotion Keyword: " + (hasPromo ? "Yes" :
"No"));
        System.out.println();
        System.out.println("Reversed Sentence: " + reversed);
        }
    }
}
```

Output :-

```
PS C:\Users\surya\OneDrive\Desktop\University\SEM4\OOPS> cd "c:\Users\surya\OneDrive\Desktop\University\SEM4\OOPS\experiment4\" ; if ($?) { javac E
xperiment42.java } ; if ($?) { java Experiment42 }
Enter feedback:
The product quality is bad but delivery was fast

Total Sentences: 1

Total Words: 9

Most Frequent Word: product

Filtered Feedback: The product quality is *** but delivery was fast

Contains Promotion Keyword: No

Reversed Sentence: fast was delivery but is quality product The
```